ASG Instance Refresh and Warm Pools

Explore the use of instance refresh and warm pools within AWS Auto Scaling groups to manage rolling updates and improve scaling speed. Understand configuration, safeguards, and combined strategies to maintain high availability and reduce downtime during deployments and traffic spikes.

Scaling applications in the cloud isn't just about adding more compute; it's about doing it wisely. When traffic spikes or application versions need rolling updates, AWS Auto Scaling groups (ASGs) provide features like instance refresh and warm pools. These two mechanisms can greatly improve the agility and resilience of the workloads. This lesson explores both, with a practical, real-world focus that aligns directly with the DVA-C02 exam objectives.

Instance refresh

Instance refresh allows us to gradually replace EC2 instances in an ASG, using a rolling update strategy. This is especially important when we update configurations, user data, or launch templates. Instead of tearing down and building everything simultaneously, Instance refresh swaps out instances individually or in small batches, minimizing service disruptions.

This mechanism is fully managed and triggered manually or automatically, such as after a new launch template version is created. AWS handles the scheduling and pacing of the updates.

Ask

Why and when to use instance refresh

Instance refresh provides a reliable solution when updating critical elements of our EC2 infrastructure, such as OS or application versions, IAM roles or instance profiles, and launch templates. Rolling out patches or configuration changes manually can be risky and disruptive, especially in production environments. These updates are applied incrementally and safely using instance refresh, preserving application uptime while enforcing configuration consistency across our fleet.

How instance refresh works internally

Instance refresh operates as a managed, stepwise process that ensures high availability and stability during updates. Below is a high-level breakdown of the sequential steps involved:

- **Snapshot of current group:** AWS captures the current configuration and state of the Auto Scaling group. This establishes a rollback baseline in case the refresh encounters issues.
- **Replace a subset of instances:** Based on user-defined parameters, such as `MinHealthyPercentage`, AWS gradually replaces a portion of instances in controlled batches.
- **Health checks:** Each new instance undergoes EC2 and ELB health checks. This ensures their integrity and operational readiness before the process moves to the next batch.
- **Rollback if failure threshold is reached:** If too many new instances fail health checks or violate policy thresholds, Auto Scaling cancels the refresh and rolls back the group to its previously stable state.

> **Note:** This is handled through the Auto Scaling group's integration with CloudWatch alarms and EC2 health status.

Ask

Configuring instance refresh

To configure instance refresh, we can initiate it either manually using the AWS CLI or SDK, or automatically using infrastructure as code (IaC) tools like AWS CloudFormation. The manual method offers fine-grained control using the `start-instance-refresh` API or CLI, while CloudFormation supports automation via the `AutoScalingRollingUpdate` policy defined in the `UpdatePolicy` section of a template.

Example using CloudFormation

Below is a simplified CloudFormation snippet demonstrating how to automatically apply instance refresh when the launch template is updated:

index.yaml YAML

```yaml
Resources:
  MyAutoScalingGroup:
    Type: AWS::AutoScaling::AutoScalingGroup
    Properties:
      LaunchTemplate:
        LaunchTemplateId: !Ref MyLaunchTemplate
        Version: !GetAtt MyLaunchTemplate.LatestVersionNumber
      MinSize: 2
      MaxSize: 5
      DesiredCapacity: 3
    UpdatePolicy:
      AutoScalingRollingUpdate:
        MinInstancesInService: 2
```

CloudFormation snippet

This configuration ensures that when the launch template changes, CloudFormation triggers an instance refresh that maintains availability by keeping a minimum number of healthy instances in service during the update.

Ask

Example in AWS CLI

The examples below show how to manually start an instance refresh using both the AWS CLI and Python SDK (Boto3):

main.sh Shell-20

```
aws autoscaling start-instance-refresh \
  --auto-scaling-group-name my-asg \            # Specifies the Auto S
  --strategy Rolling \                           # Chooses rolling up
  --preferences MinHealthyPercentage=90,InstanceWarmup=300  # Sets hea
```

CLI to manually start an instance refresh

Example in Python using Boto3

We can also trigger an Instance Refresh programmatically using the AWS SDK for Python (Boto3). This is especially useful in automated pipelines or for integrating updates into deployment workflows. The following example demonstrates how to start an Instance Refresh with specific preferences for health and warm-up behavior:

main.py C++

```
import boto3                                  # Import the Boto3 SDK
client = boto3.client('autoscaling')          # Create an Auto Sc

response = client.start_instance_refresh(     # Initiates the instanc
    AutoScalingGroupName='my-asg',            # Name of the target Au
    Strategy='Rolling',                       # Use rolling update fo
```

Ask

```
        Preferences={
            'MinHealthyPercentage': 90,           # Maintain at least 90%
            'InstanceWarmup': 300                 # Allow 300 seconds for
        }
    )
```

SDK usage to programmatically trigger an instance refresh

- **Line 8:** `MinHealthyPercentage` sets the minimum percentage of instances that must be healthy during the refresh process before the Auto Scaling group proceeds to the next batch. This ensures application availability is maintained even during updates.
- **Line 9:** `InstanceWarmup` defines the amount of time AWS waits after launching a new instance before it is considered healthy. This allows the instance to fully initialize and pass health checks before being included in the healthy capacity pool.

Safeguards: Health checks, rollbacks, and minimum healthy percentage

The mechanisms of health checks, rollbacks, and `MinHealthyPercentage` serve as essential safeguards during an instance refresh. These controls are built into the refresh workflow to ensure our Auto Scaling group maintains availability and stability while instances are being replaced.

While these topics were mentioned earlier, it's important to view them collectively as part of our risk mitigation strategy. AWS relies on EC2 and ELB health checks to validate instances before advancing to the next batch. The `MinHealthyPercentage` ensures our operational capacity never dips below a specified threshold. If issues arise, automatic rollback restores the last kno good state.

Ask

Always monitor via CloudWatch alarms and ASG activity history to catch anomalies early and confirm these safeguards function as intended.

Warm pools

A **warm pool** is a set of pre-initialized EC2 instances that are kept in a stopped or running-but-not-in-service state within an Auto Scaling group (ASG). Instead of launching new instances from scratch when scaling out, the ASG can draw instances from the warm pool, significantly reducing the time it takes for new instances to become available and handle traffic.

This section focuses on how to implement warm pools to support rapid scaling in ASG. We'll learn how to configure them, understand their lifecycle, and see where they fit into predictable or startup-intensive workloads.

Implementing warm pools for faster scaling

Warm pools act as a buffer zone: pre-initialized EC2 instances are kept in a stopped or running-but-unattached state. When a scaling event occurs, these warm instances are brought into service instantly, avoiding boot delays.

- **Applications with high startup times:** Applications like Java-based platforms or containerized microservices often require significant initialization time. In such cases, scaling directly from a cold state can introduce critical latency. Warm pools mitigate this by staging instances in advance, so when demand increases, these pre-initialized instances are pulled in almost instantly, delivering a smoother user experience.
- **Predictable bursts:** Certain workloads have predictable traffic patterns like a corporate portal that experiences a login surge every weekday morning. Instead of relying solely on predictive scaling or reacting in real time, warm pools allow developers to prepare idle but ready-to-activate instances to instantly meet demand without the startup lag associated with fresh EC2 launches.

Ask

Setting up warm pools in ASGs

We can enable warm pools using the AWS Console, CLI, or SDK. The following AWS CLI command demonstrates how to configure a warm pool for an Auto Scaling group:

main.sh Shell-20

```
aws autoscaling put-warm-pool \
  --auto-scaling-group-name my-asg \        # Specify the Auto Scalin
  --pool-state Stopped \                     # Keep warm instances in
  --max-group-prepared-capacity 5 \          # Max number of instance
  --min-size 2                               # Maintain at least 2 wa
```

CLI Example

This command sets up a warm pool with the following configurations:

- **`pool-state`:** This parameter determines whether the pre-initialized instances in the warm pool should remain in a `Stopped` state (to save costs) or a `Running` state (for even faster readiness). Stopped instances reduce operational costs, but they take slightly more time to resume when scaling events occur.
- **`max-group-prepared-capacity`:** This sets the maximum number of instances (both in service and in the warm pool) that the Auto Scaling can maintain in total.
- **`min-size`:** This ensures a minimum baseline of instances are always available in the warm pool, ready to be activated quickly when demand

Ask

increases.

Warm pool reuse strategies and lifecycle transitions

When an instance is removed from an Auto Scaling group (ASG), AWS can place it back into the warm pool instead of terminating it. This reuse mechanism is configurable and further helps reduce launch latency.

`Pending` → `Warmed` → `InService`

This transition represents the typical onboarding of an EC2 instance into the Auto Scaling group. An instance begins in the Pending state, then gets pre-initialized into the `Warmed` state. In this state, it's either stopped or unattached but ready. Finally, it moves to the `InService` state when needed to handle active traffic. This flow minimizes boot time delays during actual scaling events.

`InService` → `Warmed` → `Terminate` or `Reuse`

When scaling down or refreshing instances, an instance might be detached from active service. Instead of terminating it immediately, AWS provides the option to place it back into the `Warmed` state. From there, it can either be reused for future scaling needs or terminated. The decision to terminate or reuse depends on lifecycle configuration and resource optimization strategies. We should decide whether to reuse instances based on our application's startup cost and statefulness.

Instance refresh with warm pools: Combined strategy

Combining instance refresh and warm pools creates a highly resilient and responsive scaling environment. While instance refresh ensures safe and consistent deployment of updated EC2 instances, warm pools keep spare capacity ready to meet demand instantly. This integrated approach is especially

Ask

beneficial when workloads require both seamless software rollouts and immediate scale-out responsiveness.

- **Use instance refresh to roll out updates:** Suppose we're releasing a new version of our backend service. Rather than swapping out all EC2 instances at once, we can use instance refresh to perform a rolling update overnight. This ensures any issues are caught early and the bulk of our fleet remains available to serve requests. This strategy is invaluable when deploying sensitive changes or security patches during off-peak hours.
- **Retain pre-warmed instances for daytime scaling:** Imagine our application experiences a predictable load spike at 9 a.m. every weekday. With warm pools configured, we have several pre-initialized instances staged and ready to activate instantly when traffic increases. This ensures a seamless user experience, especially in customer-facing applications like e-commerce portals or enterprise dashboards.

This hybrid design leverages both capabilities: instance refresh for reliability and warm pools for speed, ensuring agility without sacrificing uptime or performance.

Real-world scenarios and best practices

This section illustrates how combining instance refresh and warm pools helps solve everyday challenges in different application contexts. These examples show how AWS developers can maintain uptime, speed up deployments, and optimize cost-efficiency while preparing for usage bursts or iterative rollouts.

- **E-commerce deployment:** In online retail, uptime and responsiveness directly affect revenue. Consider a retailer applying a new AMI every night to improve performance or fix security vulnerabilities. Using instance refresh, they can roll out these updates gradually during off-peak hours without disrupting customers. The next day, warm pools ensure rapid scaling during flash sales or marketing campaigns, maintaining high availability and performance.

Ask

- **CI/CD pipelines:** Development teams that practice continuous delivery often deploy updates multiple times per day. By combining AWS CodeDeploy with warm pools, they can stage EC2 instances ahead of time, allowing code to be pushed and tested rapidly. This setup minimizes wait times, increases test environment readiness, and supports faster iteration without sacrificing reliability.
- **Avoid scaling lag:** Some applications, like video rendering platforms or analytical batch processors, have compute-heavy workloads that require several minutes to initialize. Waiting for EC2 instances to launch during demand spikes could delay user jobs. Warm pools eliminate this lag by keeping ready-to-go instances on standby, improving both speed and user satisfaction.

> **Tip:** Always validate that our warm instances can rejoin without session conflicts or security token expirations.

Conclusion

Instance refresh and warm pools streamline EC2 management by enabling safe, rolling updates and reducing scale-out latency. While powerful, these features have limitations such as warm pool EBS costs and unsupported use with Spot-only ASGs. Understanding these tools in context helps us implement reliable and

Ask